

Real-Time Weapon Detection System for Surveillance Video Footages - a Preliminary Approach

Pedro Monteiro
DETI, Universidade de Aveiro
Aveiro, Portugal
pmapm@ua.pt

Gonalo Carnaz
IEETA, ISCAA, Universidade de Aveiro
Aveiro, Portugal
goncalo.carnaz@ua.pt

Osvaldo Rocha Pacheco
IEETA, DETI, Universidade de Aveiro
Aveiro, Portugal
orp@ua.pt

Abstract — The detection of weapons supported by deep learning algorithms is a research topic and application in the domains of public safety, law enforcement, security, and defense. The motivations lie in the increase in crimes with the use of weapons, near schools or other institutions. Therefore, closed circuit television (CCTV) systems play a key role in security monitoring, however, some gaps are enumerated, such as the requirement of human labor to supervise the area, which is a repetitive and time-consuming task. Previous works have faced several hurdles, especially in instances of poor weapon visibility and low-quality imagery. This approach aims to address the enumerated challenges by improving the capability for real-time detection of weapons within CCTV systems. The objective is to create an end-to-end system, named as SafeGuard, that assists law enforcement in pinpointing criminal behavior on online and offline video footage. This approach aims to improve model detection accuracy, making them more effective and resulting in better security. The preliminary metrics results of the proposed machine learning model achieved an Accuracy of 0.91.

Keywords - CCTV; surveillance; real-time; weapon; detection; YOLOv5; deep learning; security; monitoring; criminal activity.

I. INTRODUCTION

In recent years, firearm-related crimes have increased, with a wide range of incidents from different types, such as mass shootings, active shooter situations, and others. In 2022, the Federal Bureau of Investigation (FBI) [1] reported fifty accidents with active shooters in twenty-five states that resulted in three hundred thirteen casualties, as shown in figure 1.

The amount of civilian gun ownership in the United States has a ratio of 120.5 firearms for every one hundred residents, being rifles and handguns the most used [1]. Despite strict gun laws, acquiring firearms remains relatively easy in the United States. The reasons for this amount of civilian gun ownership in is an interesting field of research, however is out of scope for our research.

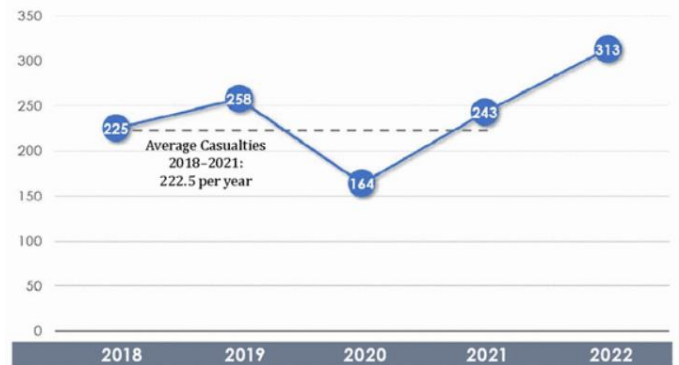


Figure 1. Active shooter incidents casualties in the US.

Regarding Portugal, the RASI report [2] reveals that crime increased by 14.1% in 2022, reaching 343845 crime reports, and by 14.4% to 13281 concerning violent and serious crimes. Figure 2 illustrates the crime rates in Portugal from 2014 to 2022 and evaluates its progression over this period. However, between 2019 (final) and 2021, the pandemic years, there was a significant decrease in violent and serious crimes.

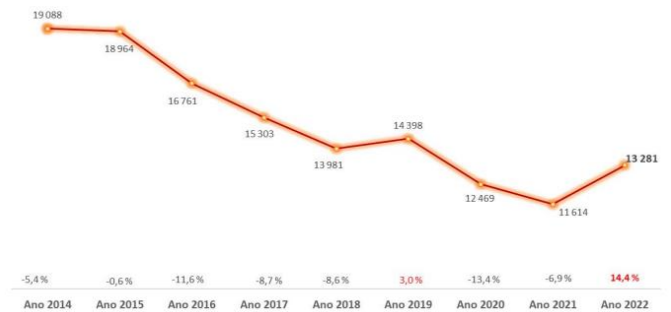


Figure 2. Crime Rates in Portugal from 2014 to 2022.

To prevent crimes, the LEAs (Law Enforcement Agencies) proposed different approaches, such as, the use of CCTV

(Closed Circuit Television) systems in public environments. For example, in Florida (USA) the use of CCTVs results in a decrease of 51% in crimes in certain areas [3].

The use of CCTV systems for crime prevention and detection raises some points to be discussed, like:

- Effectiveness and efficiency: The effectiveness of CCTV systems in detecting certain types of crime, such as car theft, is being debated [4];
- Privacy: these systems are installed in public spaces, leading to the invasion of people's privacy, which can be an obstacle in some countries, even if the safety of citizens is at stake;
- Human monitoring: most of these systems require human usage, which can lead to human error, motivated by the lack of attention or fatigue;
- Location and crime type: The impact of CCTV can vary significantly depending on the location and the type of crime. For example, CCTV might be more effective at preventing property crimes (like burglary) than it is at preventing violent crimes.

The remainder of this paper has been structured as follows:

- Section II provides an overview of the contextual background for firearms and melee weapons, as well as detailing the technical specifications, setup, and configuration of CCTV systems;
- Section III shifts focus to a critical examination of related work, reviewing existing literature and research in the field;
- Section IV discusses the SafeGuard architecture and its modules including detailed descriptions of the frontend, backend, dataset, and the object detection model employed;
- Section V presents the experimental results, highlighting the performance of the model across various metrics;
- Finally, section VI summarizes the key findings and identifies topics for further study.

II. CONTEXT

This section describes the concepts related to the study domain. We divided it into two sections: the concept of weapons and the CCTV system.

A. Weapons

According to the Cambridge Dictionary [5], a weapon is defined as an item employed in conflict or warfare that can cause injury or damage to living beings, buildings, or systems. There are different kinds of weapons, however, for our approach, we distinguish between two types: firearms and melee weapons. The melee weapons are used in close combat, requiring the user to be within immediate reach of the target. Examples include swords, daggers, and maces. Firearms, a category of ranged weapons, launch projectiles using the force generated by the rapid expansion of high-pressure gas from chemical reactions. They are divided into several subcategories:

- Handguns: Compact firearms such as pistols and revolvers that can be operated with one hand;

- Rifles: These are designed for precision, featuring a longer barrel for improved accuracy and are used in situations where precise targeting is necessary;
- Shotguns: Firearms that shoot shells filled with multiple pellets, effective in close-range scenarios;
- Automatic and Semi-Automatic Firearms: Automatic weapons discharge projectiles continuously with the trigger depressed, whereas semi-automatic weapons release one round per trigger pull.

B. CCTV

CCTV applies the use of cameras placed in strategic locations to capture video footage, as surveillance technology (a continuous monitoring activity). The images are captured in real-time and sent to monitoring screens for immediate viewing or future reference. These cameras are placed in key locations to create a grid of areas that are under surveillance. CCTV users can decide on the images that are displayed on screens. These systems are capable of triggering alerts about potential threats, particularly during periods when facilities are not in use.

Following this, enhancing a CCTV system's effectiveness involves several critical considerations. Analyzing the environmental context of the installation site, including local crime patterns, is essential for the strategic placement of cameras in high-risk zones. Ensuring an unobstructed line of sight by removing obstacles and regular maintenance is crucial. The choice of camera design, like semi-covert 'dome' models, can also significantly deter criminal activities. Lastly, an efficient CCTV setup requires not just the initial installation but continuous monitoring and the capacity for immediate enforcement actions.

III. RELATED WORK

This section summarizes previous works that are relevant to our research topic to establish a baseline of existing knowledge and to delineate the new directions that follow. The works were selected by the following criteria: the use of CCTV as an acquire camera; the use of weapons similar to our research; the deep learning models are based on Yolov5 or other versions; the publication date started at 2021 until 2024.

Rehman and Fahad [6] introduced an automated weapon detection system supported by the YOLOv5 deep learning model with an F-Score metric of 0.95 when applied to CCTV footage. The authors used a dataset divided into three categories: short guns, long guns, and knives, including videos from YouTube, simulations generated within the Unity framework, and images sourced from the Open Images Dataset [7] and Kaggle [8]. To achieve better results, the video was divided into frames and labeled using the LabelImg Python utility, also, data augmentation strategies were applied.

Thangaraj et al. [9] developed a weapon detection system to address the challenge of detecting hidden weapons in public spaces, using the YOLOv5 deep learning framework within the PyTorch framework and comparing it with a previous version of YOLO, version 3. The generated model was trained using 1496 images with COCO dataset weights, divided into 1200 for training and 296 for validation/testing. The metric mAP achieved a value of 0.92. The epoch training set was a value of

40, which takes around 120 minutes for 416x416 pixel images. The handgun detection reveals better results on YOLOv5.

Bhatti et al. [10] proposed a set of datasets for real-time pistol detection, pooling data from various online platforms, such as the Internet Movie Firearm Database [11]. The datasets enabled the distinguishing of actual handguns from commonly mistaken objects, like wallets and cell phones, with a focus on standardizing image characteristics and employing data augmentation to improve the model's adaptability. The presented study refers to the complexities of real-time weapon detection challenges such as obscured angles and concealed firearms. A broad array of sources and algorithms, including VGG16, Inception models, and YOLO versions, YOLOv4 was identified as the most effective, achieving an F1-score of 0.91 and a mean average precision of 0.92.

Mukto et Al. [12] presented a Crime Monitoring System (CMS) that utilizes surveillance cameras for real-time crime detection, aiming to overcome human limitations, such as distraction and delayed reactions. Utilizing YOLOv5 for weapon detection and MobileNetv2 for violence detection, alongside face recognition algorithms, the CMS showed over 0.80 accuracy in weapon detection, 0.95 in violence detection, and 0.97 in face recognition. Therefore, for evaluation, real-life scenarios were used. As a result, CMS efficiently detects criminal activities and alerts law enforcement.

Al-Mousa et al. [13] implemented a security system using a Raspberry Pi B+ with a camera at a lab entrance, capturing images of students with handgun replicas, collecting an initial dataset of 14000 images, which doubled to 28000 after data augmentation. They noted a challenge with detecting small guns, which could lead to misclassification. The process involved image capture, preprocessing, and analysis by a Deep Neural Network for weapon detection, sending alerts and images to a smartphone upon detection. The system achieved a 0.92 accuracy and could detect threats in 1.6 seconds.

Safa and M [14] identified challenges in dataset creation due to the lack of properly labeled data for training classifiers. To address this, 6000 images of pistols, knives, and screwdrivers were web-scraped and manually labeled using LabelImg and Roboflow [15], and then normalized to 416x416 resolution. The dataset split was 80:20, employing YOLOv8. The authors improved monitoring efficiency, reaching a 0.93 accuracy on CCTV footage.

The research showcases diverse deep learning strategies for weapon detection in various environments. Rehman and Fahad and Safa and M achieved detection accuracies of 0.95 and 0.93, respectively, using YOLO on web-sourced and CCTV footage. Thangaraj et al. and Al-Mousa et al. both reached a mean average precision (mAP) of 0.92, with Al-Mousa's system detecting threats in just 1.6 seconds. Bhatti et al. refined real-time detection with an F1-score of 0.91, distinguishing handguns from similar items. Mukto et al. furthered their scope to a Crime Monitoring System, detecting weapons with an accuracy of 0.80 and helping in violence and face recognition.

IV. SAFEGUARD SYSTEM

This section describes the architecture of the SafeGuard system (see figure 3), where the entry point (1) consists of video

feeds from CCTV cameras, which are directed to a queue system (2) that manages the flow of frames to the back-end. This backend (3) is the system core, employing an object detection model (3.1) that analyzes the video content and subsequently transmits the relevant data to the frontend. A crucial component of this architecture is dedicated to enabling real-time triggers upon the detection of objects, ensuring timely responses to significant events.

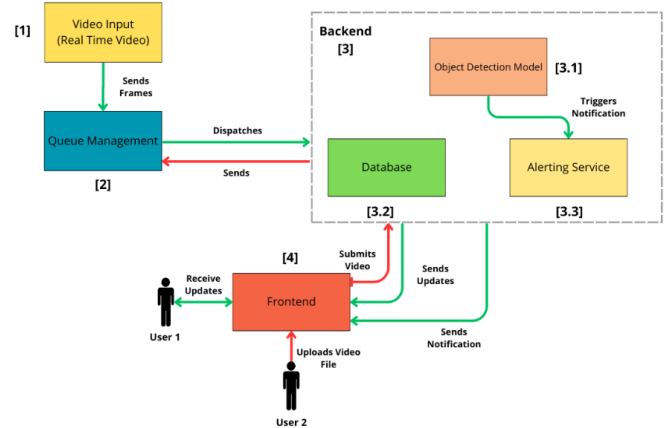


Figure 3. SafeGuard - High Level Architecture.

The frontend (4) enhances user interaction and provides real-time insights into the video analysis process. It is designed to display the video feeds being processed by the backend, overlaying the detected objects with bounding boxes directly on the video stream.

The main components of this system will be detailed in the sections that follow, providing a comprehensive overview of its operational mechanics.

A. Frontend

A key functionality is its dedicated single-camera view, figure 4, offering live video feeds with real-time detection. As the video is streamed, the model actively scans for objects, and upon detection, a bounding box appears on the feed to highlight the identified object. Also, it features a table displaying detected objects, such as weapons, along with their timestamps. An alert section is also included to notify users of any detected anomalies, ensuring prompt attention to potential security threats.

The multi-camera view offers a comprehensive monitoring solution by displaying live footage from multiple cameras simultaneously. This enables users to oversee extensive areas effortlessly and navigate between different video feeds without compromising surveillance effectiveness. Detection alerts from all feeds are integrated into this view.

Lastly, the application features a video upload page, significantly extending its functionality beyond mere live monitoring. This view allows users to submit pre-recorded video content for thorough object detection analysis. It addresses the crucial requirement for conducting detailed security reviews after the fact and facilitates the collection of evidence subsequent to incidents.

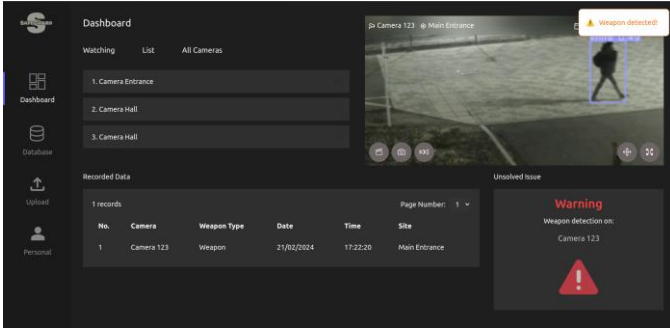


Figure 4. Frontend Single-Camera View.

Developed using the React framework, uses React's strengths, such as reusable UI components, to create a dynamic and efficient user interface. It features a dark mode design for modern aesthetics and improved accessibility, reducing eye strain.

B. Backend

The backend system architecture enables the interface between the CCTV system (online) or the storage video (offline), and the end user. It is divided into three modules:

- Object Detection Module: enables the detection of weapons based on a trained model (more detailed description in section IV-D);
- Alerting system: after the previous module detects a suspicious event, an alert message is sent to the end user. This module enables this action;
- Database: responsible to store in a permanent way the data related to a certain time-frame, storing users actions, the detection events or other anomalous situations;

The core of the backend is the integration of a pre-trained YOLOv5 model. This aspect, which will receive a more detailed exploration in the subsequent section, is critical for the system's ability to perform real-time object detection with high precision and speed.

Through designed endpoints, the backend offers functionality to retrieve information on available cameras and to store frames where detections occur. This ensures that the system not only provides real-time surveillance data but also maintains a record of significant events.

The application uses WebSockets for the real-time transmission of video and weapon detection alerts. This technology choice ensures minimal latency, facilitating an immediate response to detected threats. Furthermore, WebSockets enable the backend to trigger alarms in the frontend, alerting users to potential security breaches instantly.

The backend architecture is constructed using Django, a high-level Python web framework that emphasizes rapid development and clean design.

C. Dataset

In this subsection, we describe the proposed dataset for training and testing the detection model. Including the details about the size, composition, and diversity of the images.

To train the proposed model, a dataset was built by merging, randomly, two datasets. The main objective is to create a comprehensive collection of images that simulate a wide range of security scenarios, that exist in the source datasets:

- the Mock Attack Dataset [16];
- the DaSCI Dataset [17].

We selected these datasets because of their availability for download, which is a constraint on this application domain regarding user privacy that reduces the datasets available for public use. Furthermore, the datasets include images from various scenarios and feature different weapons, offering a strong base for our model development.

Therefore, the Mock Attack dataset consists of images captured by three surveillance cameras positioned in strategic locations, like corridors and an entrance. It offers a variety of angles showcasing scenarios such as individuals wielding weapons. The frames were annotated at a rate of two frames per second to realistically portray the scenarios. The dataset is detailed as follows:

- 607 frames from a corridor with consistent lighting;
- 3511 frames from another corridor with complex surroundings;
- 1031 frames from an entrance with varied lighting and obstacles.

Figure 5 shows an example of an image taken in a corridor. A person is walking along the corridor using a weapon in his right hand.



Figure 5. Mock Attack Dataset [16] Cam1 Frame.

The DaSCI dataset is the other collection used. This selection features both weapons and everyday items like wallets or bicycles, aimed at enhancing the dataset's diversity. Originally, the DaSCI Dataset included six different classes of objects. Therefore, we reformulated these classes into two categories: weapons and knives. Also, the annotations for these images were converted from .xml to .txt format to ensure compatibility with the YOLOv5 object detection algorithm.

Finally, our dataset is a result of the combination of both the Mock Attack and DaSCI datasets, which were subjected to random sampling to generate a training and testing split. This

split followed an 80:20 ratio, allocating 80% of the images for training the object detection algorithm and 20% for testing its performance. In total, it contains 11020 images, as detailed in Table I.

TABLE I. SOURCE DATASETS SUMMARY

Src	Imgs	Resolution	Format	Desc.
DaSCI	5371	*	jpg	Variety of weapons displayed in different positions
	500	*	jpg	Random everyday objects
Mock Attack	607	1920x1080	png	Cam1: focus on uniform lighting and some conflicting objects such as doors or bins.
	3511	1920x1080	png	Cam5: features objects like a black carpet on the floor, with irregular lighting.
	1031	1920x1080	png	Cam7: similar to Cam1 but with more conflicting objects.

* Multiple Formats

D. Object Detection Model

The Object Detection Model enables the detection of the weapons present in images acquired by CCTV cameras. To perform this task, we developed a deep-learning model using the YOLOv5 algorithm. Figure 6 shows the Pre-Trained Model, which was generated to achieve great performance in identifying and classifying weapons, after a comprehensive process of training, testing, and evaluation.

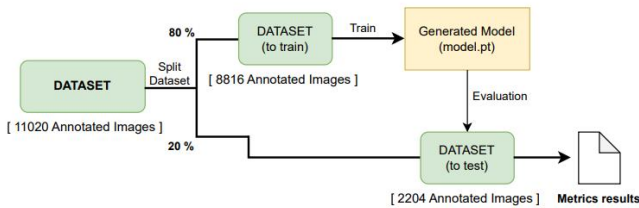


Figure 6. Model Flow Overview.

YOLOv5 was selected due to its optimal balance of speed and accuracy, leveraging the strengths of previous YOLO versions to achieve improved detection performance. Its demonstrated stability and maturity, validated through extensive real-world testing and a comprehensive support ecosystem, further establish YOLOv5 as a reliable choice. While newer iterations may introduce specific advancements, they lack the established reliability and resource availability that YOLOv5 boasts, making YOLOv5 the preferred option for dependable and efficient object detection tasks.

For training and evaluation purposes, we proposed a script that performed the following steps (high-level description):

- The script begins by checking if CUDA is available; if so, it uses CUDA to accelerate the training process;
- It then proceeds to download from a pre-organized repository the necessary files for training, detection,

and evaluation of the object detection model, as well as the dataset;

- Having obtained the necessary scripts and the dataset, the script executes a function to train the YOLOv5 model. It is designed to support multiple values for epochs, batch size, and image size, allowing for flexible adjustments to optimize training outcomes;
- Following the training phase, the script uses the trained model to detect objects, subsequently evaluating the model's performance. This evaluation includes generating metrics such as the Confusion Matrix, and plots for Accuracy, F1-score, Recall, and Intersection over Union;
- All steps are timed, and both the durations and the parameters used are recorded in a logs file.

The model used the proposed dataset for training and testing. This custom dataset played a crucial role in fine-tuning the model, allowing it to learn and adapt to the unique characteristics of the surveillance environment.

V. EXPERIMENTAL RESULTS

This section presents the experimental results achieved by the proposed deep learning model (described in section IV-D) for weapon and knife object detection over the proposed dataset. To achieve these results, we conduct the analysis automated by a script (already discussed) that retrieves an array of metrics for each trial, delivering an in-depth evaluation of the model's effectiveness across various parameter settings, as detailed in Table II.

TABLE II. MODEL PERFORMANCE METRICS

Eps	BSize	ISize	Accuracy	F1-score	Precision	Recall
50	16	640	0.896	0.87	0.889	0.845
50	32	256	0.729	0.73	0.855	0.649
50	32	640	0.892	0.87	0.901	0.833
80	16	640	0.899	0.87	0.901	0.837
80	32	256	0.756	0.76	0.873	0.692
80	32	640	0.905	0.87	0.901	0.848
100	16	640	0.903	0.88	0.905	0.855
100	32	256	0.760	0.76	0.859	0.692
100	32	640	0.905	0.88	0.919	0.850
120	16	640	0.910	0.88	0.922	0.852
120	32	640	0.906	0.88	0.921	0.835

The table lists the results of models trained for different numbers of epochs (Eps), batch sizes (BSize), and image sizes (ISize), evaluating them on four key metrics: Accuracy, F1-score, Precision, and Recall.

The model trained for 120 epochs with a batch size of 16 and an image size of 640 shows the highest accuracy of 0.910. This model also has a high F1-score of 0.88, Precision of 0.922, and Recall of 0.852. A high Precision indicates a low false

positive rate, while a high Recall indicates a low false negative rate, critical in weapon detection scenarios as it reflects the model's capability to correctly identify true positives, the actual presence of weapons.

Comparing across epochs, we notice that models with 120 epochs consistently outperform those with 50 or 80 epochs, regardless of the batch size and image size.

Furthermore, when examining the effect of batch size, the models with a batch size of 16 consistently overcome the models with a batch size of 32 at the same epoch and image size settings. This suggests that a smaller batch size might be contributing to a more effective learning process.

Lastly, input size seems to have a substantial impact on model performance as well. Models with an input size of 640 generally outperform those with an input size of 256, as evidenced by the higher scores across all metrics.

In summary, the model trained for 120 epochs, with a batch size of 16 and an image size of 640, is the best-performing model according to Table II. It achieves the highest accuracy, F1-score, and precision, along with a high recall, indicating a good model with a strong generalization capability on the data it was tested on. Figure 7 illustrates the F1-Confidence curve and shows that the best F1-score of 0.88 was achieved at a confidence threshold of 0.355.

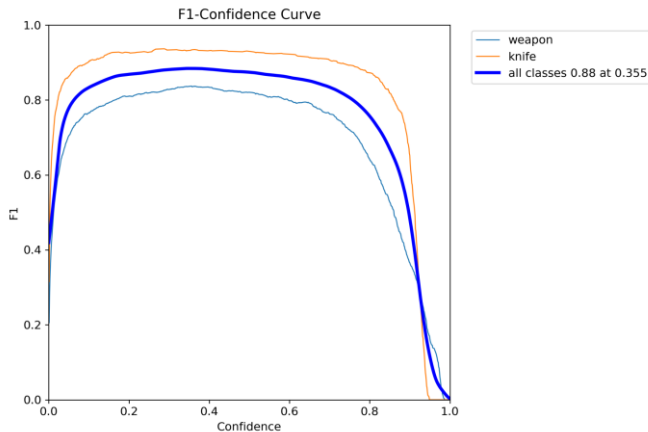


Figure 7. F1-Confidence Curve for 120 Epochs and 16 Batch Size.

Our model's performance aligns well with the results reported in section III (Related Work). Specifically, it closely matches the 0.93 accuracy rate achieved by Safa and M using YOLOv8 and the F1-score of 0.91 reported by Bhatti et al. with YOLOv4. Furthermore, Thangaraj et al. and Al-Mousa et al. reported mean Average Precision (mAP) and Accuracy values of 0.92, which are in line with our model's precision and accuracy.

VI. CONCLUSION AND FUTURE WORK

In conclusion, the proposed approach aims to achieve an end-to-end system that enables law enforcement officers to

reduce the probability of error on weapon detection on CCTV video footage by using a deep learning model for weapons detection and sending warnings to users. The system also simplifies the task for those monitoring the feeds, by removing the need for continuous manual monitoring of video footage. Through the automation of the detection process and the subsequent triggering of alerts, the system ensures a swift and accurate response to potential threats.

This work is at a preliminary stage, although the results have been obtained and the basic requirements implemented. These results allow us to understand the domain, its shortcomings, and challenges. For future work, we identify the following items:

- Improve the dataset to detect complex scenarios, such as inadequate lighting or adverse visibility conditions;
- Achieve diversity on the dataset by using data augmentation techniques;
- Implement cross-validation techniques to enhance the dataset-splitting process;
- Improve the accuracy of the deep learning model;
- Optimize the web application to display CCTV footage with minimal delay, aiming for real-time video and detection;
- Explore and analyze other versions of the YOLO architecture to explore potential improvements in detection accuracy and speed, and their integration within the existing system architecture.

REFERENCES

- [1] FBI, "Active shooter incidents in the united states in 2022", April 2023
- [2] Secretário-Geral do Sistema de Segurança Interna, "Relatório Anual de Segurança Interna", September 2022
- [3] S. C. Daily, "The effectiveness of cctv in reducing crime: Florida's experience explained", November 2023
- [4] Joseph Ramses Méndez Cam, Félix Melchor Santos López, Víctor Genaro Rosales Urbano & Eulogio Guillermo Santos de la Cruz, "Prediction Models for Car Theft Detection Using CCTV Cameras and Machine Learning: A Systematic Review of the Literature", May 2023
- [5] Cambridge Dictionary, "https://dictionary.cambridge.org/"
- [6] Abdul Rehman and Labiba Gillani Fahad, "Real-Time Detection of Knives and Firearms using Deep Learning", December 2022
- [7] Open Images Dataset, "https://storage.googleapis.com/openimages/web/index.html"
- [8] Kaggle, "https://www.kaggle.com/"
- [9] Rajasekaran Thangaraj, Pavithra T, Pandiyan P, Uma Rani M and Balasubramaniam vadivelu, "Real-Time Handgun Detection in Surveillance Videos based on Deep Learning Approach", May 2022
- [10] Muhammad Tahir Bhatti, Muhammad Gufran Khan, Masood Aslam and Muhammad Junaid Fiaz, "Weapon Detection in Real-Time CCTV Videos using Deep Learning", March 2021
- [11] Internet Movie Firearm DB, "https://imfdb.org/wiki/Main_Page"
- [12] Md. Mukhtadir Mukto, Mahamudul Hasan, Md. Maiyaz Al Mahmud, Ikramul Haque, Md. Ahsan Ahmed, Taskeed Jabid, Md. Sawkat Ali, Mohammad Rifat Ahmmad Rashid, Mohammad Manzurul Islam and Maheen Islam, "Design of a real-time crime monitoring system using deep learning techniques"
- [13] A. Al-Mousa, O. Z. Alzaibaq, and Y.A. Hashyeh, "Deep learning-based real-time weapon detection system", August 2023
- [14] F. Safa and S. M, "Subduing crime and threat in real-time by detecting weapons using yolov8", August 2023
- [15] Roboflow, "https://roboflow.com/"
- [16] Mock Attack Dataset, "https://github.com/Deepknowledge-US/US-Real-time-gun-detection-in-CCTV-An-open-problem-dataset/blob/gh-pages/index.md", accessed on October 2023
- [17] DaSCI Dataset, "https://dasci.es/", accessed on October 2023